Lab 1: Linear and logistic regression from scratch

Joshai Bensoussan

Mathematics & Statistics for Machine Learning (MSA-DATI07-04), Albert School

October 2026

Introduction

The goal of this lab was to build the two basic models of supervised learning by hand, with NumPy only, and to see that they give the same answer as the library implementations. Part A uses the Kaggle medical-cost dataset, where the target (yearly charges) is a number, so it is a regression problem. Part B uses the Titanic dataset, where the target (survived or not) is binary, so it is a classification problem.

Both parts follow one pipeline: random train/test split, pre-processing fitted on the training set only, model, cost, analytic gradient checked against finite differences, gradient descent, and finally evaluation on the test set against a naive baseline and against scikit-learn. Every checkpoint of the notebook passes, and since the seed is 42 everywhere, all numbers in this report can be reproduced by re-running it.

1 Part A: medical costs (linear regression)

A1. Exploring the data

There are 1,338 clients described by age, sex, BMI, number of children, smoking status and region. Charges range from about 1,100 to 63,800 USD. The mean is 13,270 USD but the median is only 9,382 USD, which already says the distribution is not symmetric.

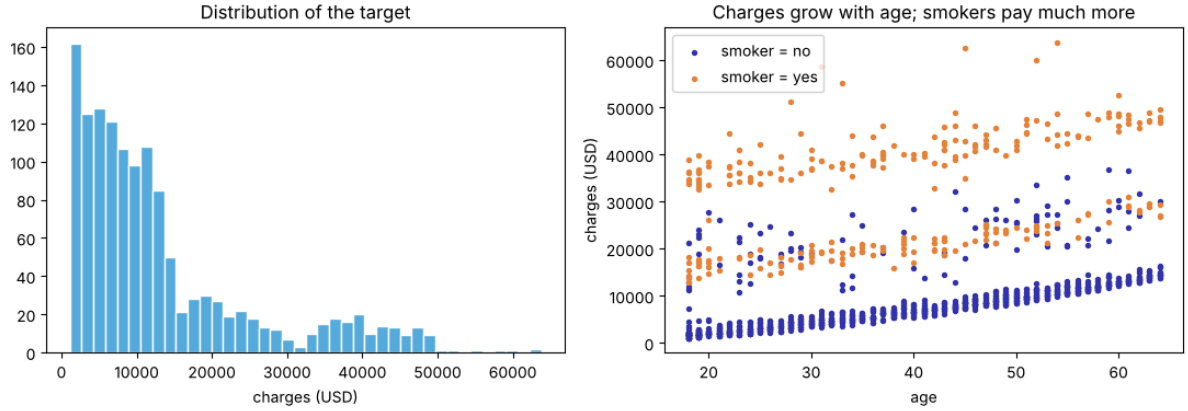


Figure 1: Distribution of the target (left) and charges against age, coloured by smoking status (right).

Answer A1. The feature that stands out is `smoker`. For a given age, smokers sit on a completely different level from non-smokers, often 20,000 to 40,000 USD higher, so the scatter plot shows separate bands rather than one cloud. Age is the second driver: inside each band, charges rise steadily with age. The target is clearly right-skewed (skewness about 1.5). Most clients cost less than 15,000 USD, and a long tail of expensive clients pulls the mean above the median.

A2. Pre-processing

The categorical columns (`sex`, `smoker`, `region`) are one-hot encoded with `drop_first=True`, which gives 8 numerical features. One level per category is dropped because otherwise the dummies of a category sum to 1 and become perfectly collinear with the intercept (the dummy-variable trap).

My split function draws a random permutation of the row indices with `np.random.default_rng(seed).permutation(n)`, sends the first 20% to the test set and keeps the remaining 80% for training (1,070 / 268 rows). Shuffling matters because a file can be sorted by some variable. A split taken in file order would then give a test set that does not follow the same distribution as the training set.

For standardisation, the mean and standard deviation are estimated on the training rows only and then applied to both sets. Using test statistics would let information from data that is supposed to be unseen leak into the model. A zero standard deviation is replaced by 1 so that a constant column cannot cause a division by zero.

```
mu = X_train.mean(axis=0)
sigma = np.where(X_train.std(axis=0) == 0, 1.0, X_train.std(axis=0))
return (X_train - mu) / sigma, (X_test - mu) / sigma
```

A3 and A4. Model, cost and gradient

The model is $\hat{y} = Xw + b$ and the cost is the halved mean squared error

$$J\left(w, b\right) = \frac{1}{2m} \sum_{i=1}^{m} \left(\hat{y}_i - y_i\right)^2 .$$

Both functions are written with matrix products only, with no loop over clients. As a sanity check, the all-zero model gives $J = 165{,}004{,}136$, which is exactly $\text{mean}\left(y^2\right) / 2$. Differentiating gives

$$\frac{\partial J}{\partial w} = \frac{1}{m} X^{\top}\left(\hat{y} - y\right), \qquad \frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^{m} \left(\hat{y}_i - y_i\right),$$

that is, the prediction error multiplied by each feature and averaged over clients. A bug in a gradient does not crash anything, it only makes the descent converge to the wrong point, so I compared it with central finite differences at a random point. The relative error is 1.6×10^{-8}, so the analytic gradient is correct.

A5. Gradient descent

The loop starts from $w = 0, b = 0$, records the cost, computes the gradient and updates w and b together. The cost and gradient functions are passed as arguments, so the same function is reused with-

out any change in Part B. With $\alpha = 0.1$ and 500 iterations the cost decreases at every step, down to 19,020,160.

For the learning-rate experiment I ran 100 iterations for four values of α. The largest eigenvalue of $X^\top X / m$ is $L \approx 1.509$, so the theory predicts divergence for $\alpha > 2 / L \approx 1.325$.

α	Cost after 100 iterations	Behaviour
0.001	138,566,389	far too slow
0.1	19,020,304	smooth convergence
1.2	19,020,160	converges, fastest of the four
1.4	6.2×10^{15}	diverges

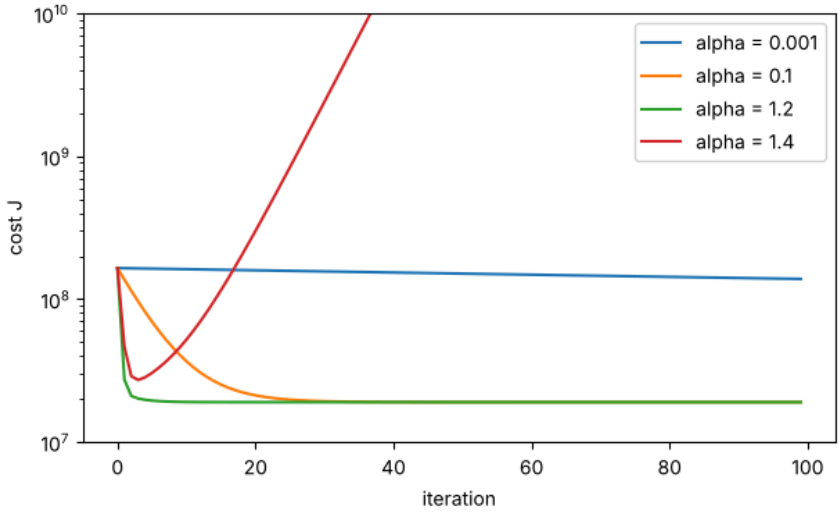


Figure 2: Learning curves for the four learning rates (log scale).

Observations. With $\alpha = 0.001$ the steps are tiny, and after 100 iterations the cost is still around 1.4×10^8, nowhere near the minimum. $\alpha = 0.1$ converges cleanly. $\alpha = 1.2$ is still below $2 / L$ and gets to the minimum even faster.

The way I understand it: for a quadratic cost, each eigendirection of $X^\top X / m$ is multiplied by $\left(1 - \alpha\lambda_j\right)$ at every step. At $\alpha = 1.2$ that factor is about -0.81 in the steepest direction, so the *weights* flip from one side of the minimum to the other at every step. Since the factor is still smaller than 1 in absolute value, the *cost* keeps decreasing; the only "increases" I found were of order 10^{-9}, which is floating-point rounding. With $\alpha = 1.4$ the factor becomes about -1.11: every step overshoots further than the previous one and the cost explodes.

So a too-small α wastes iterations and a too-large one diverges, but the best value is not right at the limit either: close to $2 / L$, the steepest direction converges very slowly again. The optimal fixed step is $2 / \left(L + \mu\right) \approx 1.09$, where $\mu \approx 0.32$ is the smallest eigenvalue, which explains why 1.2 performs so well. Standardising the features is what keeps L and μ close to each other and makes such large steps possible.

A6 and A7. Results on the test set

The final model is trained with $\alpha = 0.1$ for 2,000 iterations. As a baseline, I predict the training mean for every client.

	Train	Test	Baseline (test)
RMSE (USD)	6,168	5,526	11,184
MAE (USD)	4,305	3,955	8,322
R^2	0.750	0.756	−0.001

The model halves the error of the baseline and explains about 76% of the variance of the charges on unseen clients. Training and test scores are almost identical, so there is no sign of overfitting, which is expected with 9 parameters for 1,070 rows.

I then solved the normal equations with `np.linalg.lstsq` (adding a column of ones for the intercept) and fitted scikit-learn's `LinearRegression`. The three methods agree to three decimals on every coefficient (intercept 13,344.773, `smoker_yes` 9,699.098, …), which shows that gradient descent fully converged to the least-squares solution.

A8. Interpretation

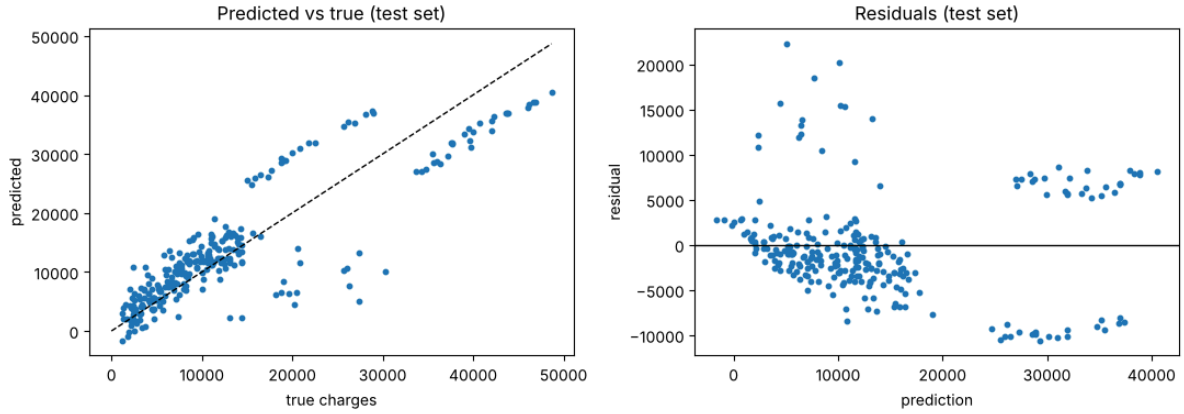


Figure 3: Predicted against true charges (left) and residuals (right) on the test set.

Question 1. Because the inputs are standardised, each weight is the effect of a one-standard-deviation increase of the feature. The largest one is `smoker_yes`, about 9,700 USD per standard deviation. This is a 0/1 variable with a standard deviation of about 0.41 on the training set, so going from non-smoker to smoker corresponds to 9,700 / 0.41 ≈ 23,800 USD. For an insurer: *a smoker costs on average about 23,800 USD more per year than a non-smoker of the same age, BMI, sex, number of children and region.* Age comes second with about 3,700 USD per standard deviation (14 years), i.e. roughly 260 USD per additional year.

Question 2. Instead of one cloud around 0, the residuals form distinct groups. The missing ingredient is the interaction between smoking and BMI. For non-smokers, BMI hardly changes the charges, whereas for smokers, being obese makes them jump. A linear model imposes one BMI slope for everybody, so it under-predicts obese smokers and over-predicts non-obese smokers, and each sub-

population ends up with its own constant bias. The A1 plot only shows age, so this hypothesis comes from domain knowledge and the residual structure rather than from that figure directly; the bonus below tests it.

Question 3 (bonus). I added the column `smoker_yes × bmi` and reran part A. Because the seed and row order are unchanged, the test set is exactly the same, so the comparison is fair.

Test set	Without interaction	With smoker × bmi
RMSE (USD)	5,526	4,225
MAE (USD)	3,955	2,712
R^2	0.756	0.857

One extra column raises the test R^2 from 0.756 to 0.857 and cuts the MAE by about a third. The limitation was the specification of the model, not the optimisation: the model is linear in its parameters, not in the raw variables, so it can capture such effects once it is given the right features.

2 Part B: Titanic (logistic regression)

B1 and B2. Data and pre-processing

Overall 38.4% of passengers survived, but this varies a lot: 74% of women against 19% of men, and 63% in first class, 47% in second and 24% in third. `Age` is missing for 177 passengers, `Embarked` for 2 (and `Cabin`, which is not used, for 687).

I kept 7 variables: `Pclass` (treated as a category, since class 3 is not "three times" class 1), `Sex`, `Age`, `SibSp`, `Parch`, `Fare` and `Embarked`. The split (713 / 178) is done first; only then are missing ages filled with the training median and missing ports with the training mode, so the test set never influences pre-processing. The test dummies are re-indexed on the training columns so both matrices have the same 9 features. The reference categories are first class, female and Cherbourg.

The split is not stratified, and the survival rate ends up at 0.369 in training against 0.444 in test. With only 178 test passengers, one passenger is worth 0.56 percentage points of accuracy, so test scores carry a few points of random noise.

B3 to B5. Sigmoid, cross-entropy and gradient

Linear regression can output values outside [0, 1], so the score $z = Xw + b$ is passed through the sigmoid $\sigma(z) = 1 / (1 + e^{-z})$ to give a survival probability. The cost is the binary cross-entropy

$$J\left(w, b\right) = -\frac{1}{m}\sum_{i=1}^{m}\left[y_i \log\hat{p}_i + \left(1 - y_i\right)\log\left(1 - \hat{p}_i\right)\right],$$

which punishes confident mistakes heavily (predicting 0.01 for a survivor costs $-\log 0.01 \approx 4.6$). Probabilities are clipped to $\left[10^{-12}, 1 - 10^{-12}\right]$ to avoid $\log 0$. With all parameters at zero, every passenger gets $\hat{p} = 0.5$ and $J = 0.6931 = \log 2$, as expected.

The nice result is that the gradient has exactly the same form as in Part A, $\frac{1}{m}X^{\top}\left(\hat{p}-y\right)$, with only the prediction changed. The finite-difference check gives a relative error of 6.8×10^{-10}.

B6. Training

The model is trained with the Part A gradient-descent function, unchanged ($\alpha = 0.5$, 2,000 iterations). I also tracked the test cost along the way: both curves decrease together and end at 0.4412 (train) and 0.4470 (test), with no point where the test cost starts rising again.

B7. Evaluation

With a threshold of 0.5, the confusion matrix on the test set is 87 true negatives, 12 false positives, 22 false negatives and 57 true positives.

	Train	Test
Accuracy	0.805	0.809
Precision	0.763	0.826
Recall	0.684	0.722
F1	0.721	0.770

Always predicting "died" already gives 0.556 accuracy on this test set, so accuracy has to be judged against that number. The model reaches 0.809. When it says "survived" it is right 83% of the time, and it finds 72% of the actual survivors.

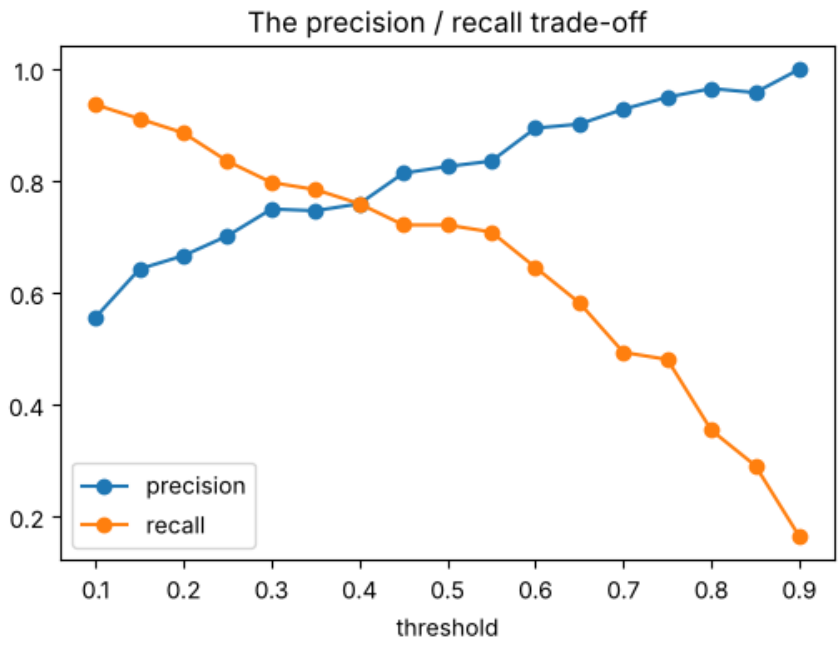


Figure 4: Precision and recall on the test set for thresholds from 0.1 to 0.9.

Lowering the threshold makes the model say "survived" more easily: recall rises (0.94 at 0.1) and precision falls (0.56, which is just the share of survivors in the test set). Raising it does the opposite:

at 0.9 every predicted survivor is correct, but only 17% of survivors are found. The two curves meet around 0.4. The threshold is therefore a business choice rather than a property of the model: a rescue team that wants to miss as few survivors as possible should work around 0.2–0.3 (recall 0.80–0.89, precision 0.67–0.75) and accept more false alarms.

B8. Comparison with scikit-learn and interpretation

`LogisticRegression(C=1e10)`, which is in practice unpenalised and therefore minimises the same cost, gives an intercept of −0.7304 against −0.7303 for my model, weights equal up to about 0.001, and the same test accuracy (0.809).

Feature	Weight	e^{w} (per std)
Sex_male	−1.285	0.277
Pclass_3	−0.936	0.392
Age	−0.476	0.621
Embarked_S	−0.291	0.747
SibSp	−0.277	0.758
Pclass_2	−0.257	0.773
Parch	−0.139	0.870
Embarked_Q	−0.109	0.897
Fare	0.114	1.121

Question 1. The strongest negative effect is being male (−1.29), then travelling in third class (−0.94), then age (−0.48). `Fare` is the only positive weight and it is small, probably because ticket price is strongly correlated with class and the class dummies already capture most of that effect. Since female and first class are the reference levels, the passengers with the best chances were women in first class. This matches "women and children first": men survive much less, and survival decreases with age. There is also a strong social-class effect on top. The age effect is probably underestimated, because the 177 missing ages were all set to the median, which flattens the relationship.

Question 2. One point needs care here. The features are standardised, so $e^{w} = 0.277$ is the change in odds for *one standard deviation* of `Sex_male` (about 0.47), not for switching from woman to man. For the actual comparison, the weight has to be divided by the standard deviation, as for `smoker_yes` in Part A: $\exp\left(-1.285 / 0.473\right) \approx 0.066$. For a non-specialist: *for two passengers with the same class, age, family and ticket, the man's odds of surviving were about 15 times smaller than the woman's.* In the same way, each extra year of age multiplied the odds by about 0.96.

Question 3. Accuracy is 0.805 on train and 0.809 on test, and the cross-entropy is 0.441 against 0.447. The two are essentially equal, so the model does not overfit, which is expected for a linear model with 10 parameters and 713 rows. The test score is even slightly higher, but that is not meaningful: it is within the noise of a 178-passenger test set whose class balance differs from the training set.

3 Going further

Log target. Fitting on $\log(\text{charges})$ and converting back with `np.exp` slightly improves the test MAE (3,821 against 3,955 USD), because the bulk of ordinary clients is fitted better. But RMSE jumps to 7,765 USD and R^2 drops to 0.517. Part of the explanation is that taking exp of a mean on the log scale estimates the *median* of the charges, not their mean, so the expensive clients are systematically under-predicted (retransformation bias). In addition, the log model turns every effect into a multiplier, which does not fit the additive smoker jump well. The log target helps the typical client but hurts on the costly tail, which RMSE weights the most.

Ridge. Adding $\frac{\lambda}{2m}\|w\|^2$ to the cost adds $\frac{\lambda}{m}w$ to the gradient of w (the intercept is not penalised), which pulls the weights towards zero at each step. With $\lambda \in \{0.1, 1, 10\}$, the norm of w only goes from 10,639 to 10,540 and the test R^2 stays at 0.756. This is what the scale predicts: the penalty acts like adding λ / m to eigenvalues between 0.32 and 1.5, and with $m = 1{,}070$ even $\lambda = 10$ adds less than 0.01. Since the model was not overfitting, there is little to gain from regularisation here.

Mini-batches. Using 32 random rows per step makes each step about 33 times cheaper than a full-batch step (1,070 rows). The full training cost decreases as fast as with full-batch descent at first, then keeps fluctuating around the minimum because each gradient is only a noisy estimate. The final test R^2 is 0.753, almost the same as 0.756. A learning rate that decreases over time would remove the remaining noise.

Conclusion

Both models built from scratch match scikit-learn (and the normal equations for linear regression), and every gradient passed the numerical check. On medical costs, the linear model explains about 76% of the variance on unseen clients, smoking is by far the dominant factor, and a single smoker × BMI interaction raises R^2 to 0.857. On the Titanic, logistic regression reaches 81% test accuracy against 56% for the majority-class baseline. Neither model overfits.

The main things I take away from this lab: pre-processing must be fitted on the training set only. The learning rate is governed by the curvature of the cost (the limit $2 / L$, and an optimum below it). Coefficients of standardised features must be converted back to original units before being explained. And the decision threshold of a classifier depends on which errors we care about most.